

16 Iterator pattern

Iterator pattern

Intent

Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Advantage

- Iterator pattern is not only about traversing through a collection, we can **provide different kind of iterators based on our requirements**.
- Iterator pattern **hides the actual implementation** of traversal through the collection and client programs just use iterator methods.

>> ช่วยอำนวยความสะดวกในการเข้าถึงข้อมูลใน Collection & ใช้ inner class ซ่อนวิธีการทำซ้ำ

Reference

[1] Iterator Design Pattern in Java – Example Tutorial

<https://www.javacodegeeks.com/2013/08/iterator-design-pattern-in-java-example-tutorial.html>

ตัวอย่าง

```
1. public enum ChannelTypeEnum {
2.     ENGLISH, HINDI, FRENCH, ALL;
3. }

1. public interface ChannelCollection {
2.     public void addChannel(Channel c);
3.     public void removeChannel(Channel c);
4.     public ChannelIterator iterator(ChannelTypeEnum type);
5. }

1. public interface ChannelIterator {
2.     public boolean hasNext();
3.     public Channel next();
4. }

1. import java.util.ArrayList;
2. import java.util.List;
3.
4. public class ChannelCollectionImpl implements ChannelCollection {
5.
6.     private List<Channel> channelsList;
7.
8.     public ChannelCollectionImpl() {
9.         channelsList = new ArrayList<>();
10.    }
11.
12.    public void addChannel(Channel c) {
13.        this.channelsList.add(c);
14.    }
15.
16.    public void removeChannel(Channel c) {
17.        this.channelsList.remove(c);
18.    }
19.
20.    @Override
21.    public ChannelIterator iterator(ChannelTypeEnum type) {
22.        return new ChannelIteratorImpl(type, this.channelsList);
23.    }
24.
25.    // INNER Class
26.    private class ChannelIteratorImpl implements ChannelIterator {
27.
28.        private ChannelTypeEnum type;
29.        private List<Channel> channels;
30.        private int position;
31.
32.        public ChannelIteratorImpl(ChannelTypeEnum ty,
33.                                   List<Channel> channelsList) {
34.            this.type = ty;
35.            this.channels = channelsList;
36.        }
37.
38.        @Override
39.        public boolean hasNext() { // implement iteration
40.            while (position < channels.size()) {
41.                Channel c = channels.get(position);
42.                if (c.getType().equals(type) ||
43.                    type.equals(ChannelTypeEnum.ALL)) {
44.                    return true;
45.                } else {
46.                    position++;
47.                }
48.            }
49.        }
50.    }
51. }
```

```

49.         return false;
50.     }
51.     @Override
52.     public Channel next() {
53.         Channel c = channels.get(position);
54.         position++;
55.         return c;
56.     }
57. }
58. }

```

```

1. public class Channel {    // simple POJO
2.
3.     private double frequency;
4.     private ChannelTypeEnum TYPE;
5.
6.     public Channel(double freq, ChannelTypeEnum type) {
7.         this.frequency = freq;
8.         this.TYPE = type;
9.     }
10.    public double getFrequency() {
11.        return frequency;
12.    }
13.    public ChannelTypeEnum getTYPE() {
14.        return TYPE;
15.    }
16.    @Override
17.    public String toString() {
18.        return "Frequency=" + this.frequency + ", Type=" + this.TYPE;
19.    }
20. }

```

```

1. public class DemoIterator {
2.
3.     public static void main(String[] args) {
4.         ChannelCollection channels = populateChannels();
5.         ChannelIterator baseIterator = channels.iterator(ChannelTypeEnum.ALL);
6.         while (baseIterator.hasNext()) {
7.             Channel c = baseIterator.next();
8.             System.out.println(c.toString());
9.         }
10.        System.out.println("*****");
11.        ChannelIterator englishIterator =
12.            channels.iterator(ChannelTypeEnum.ENGLISH);
13.        while (englishIterator.hasNext()) {
14.            Channel c = englishIterator.next();
15.            System.out.println(c.toString());
16.        }
17.    }
18.
19.    private static ChannelCollection populateChannels() {
20.        ChannelCollection channels = new ChannelCollectionImpl();
21.        channels.addChannel(new Channel(98.5, ChannelTypeEnum.ENGLISH));
22.        channels.addChannel(new Channel(99.5, ChannelTypeEnum.HINDI));
23.        channels.addChannel(new Channel(100.5, ChannelTypeEnum.FRENCH));
24.        channels.addChannel(new Channel(101.5, ChannelTypeEnum.ENGLISH));
25.        channels.addChannel(new Channel(102.5, ChannelTypeEnum.HINDI));
26.        channels.addChannel(new Channel(103.5, ChannelTypeEnum.FRENCH));
27.        channels.addChannel(new Channel(104.5, ChannelTypeEnum.ENGLISH));
28.        channels.addChannel(new Channel(105.5, ChannelTypeEnum.HINDI));
29.        channels.addChannel(new Channel(106.5, ChannelTypeEnum.FRENCH));
30.        return channels;
31.    }
32.
33. }

```

```
Output - InterpreterIterator (run)
run:
Frequency=98.5, Type=ENGLISH
Frequency=99.5, Type=HINDI
Frequency=100.5, Type=FRENCH
Frequency=101.5, Type=ENGLISH
Frequency=102.5, Type=HINDI
Frequency=103.5, Type=FRENCH
Frequency=104.5, Type=ENGLISH
Frequency=105.5, Type=HINDI
Frequency=106.5, Type=FRENCH
*****
Frequency=98.5, Type=ENGLISH
Frequency=101.5, Type=ENGLISH
Frequency=104.5, Type=ENGLISH
BUILD SUCCESSFUL (total time: 0 seconds)
```